



مبانی برنامه نویسی

Functions

Dr. Mehran Safayani

safayani@iut.ac.ir

safayani.iut.ac.ir



<https://www.aparat.com/mehran.safayani>



https://github.com/safayani/Programming_Basics_course



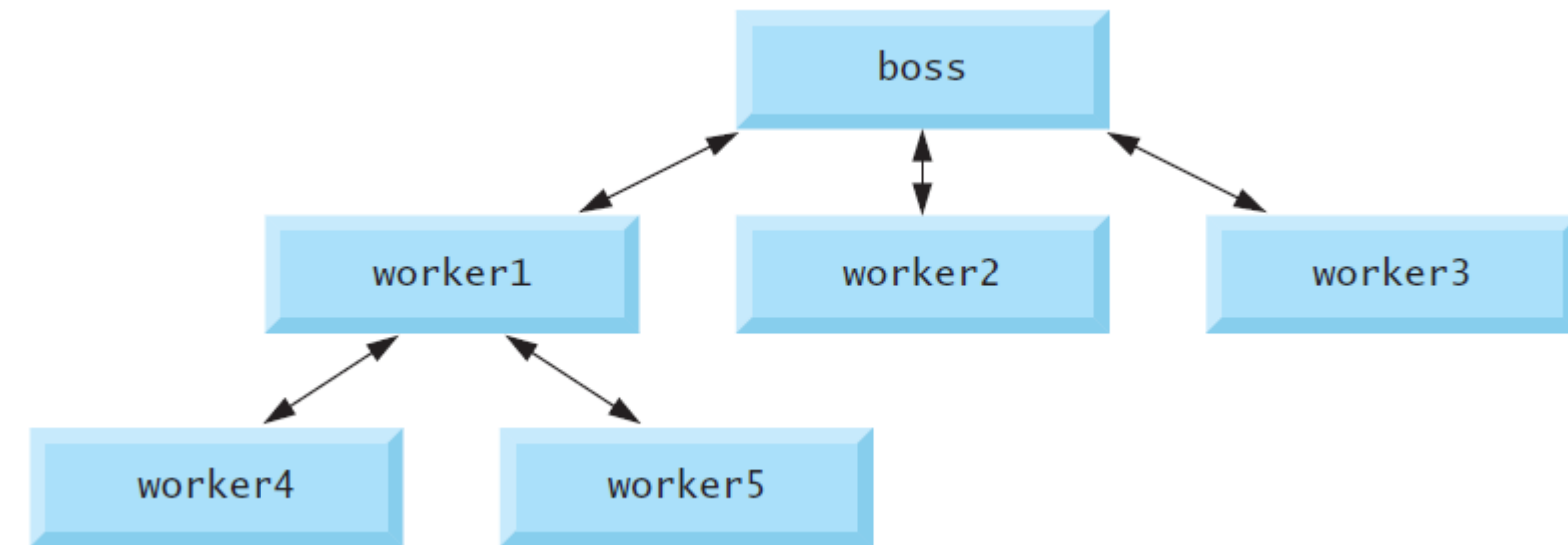
Managing Program Complexity

- **The Challenge:** Real-world programs are large and complex.
- **The Strategy:** Use the “**Divide and Conquer**” technique.
 - Break down large programs into smaller, more manageable pieces.
- **The Goal:** Simplify development, operation, and maintenance.

Modular Programming in C with Functions

- **The Tool:** C uses **functions** to modularize programs.
- **Implementation:** Combine two types of functions:
 - New functions written by the programmer.
 - Pre-packaged functions from the **C Standard Library**.
- **C Standard Library:** A rich collection of ready-to-use functions for:
 - Mathematical calculations (pow)
 - String and character manipulation
 - Input/Output (printf, scanf)
 - And many other common operations.

Defining Functions



- **Purpose:** Define functions to perform specific, reusable tasks.
 - **Efficiency:** Write code once, use it multiple times.
 - **Information Hiding:** A function’s internal logic is hidden from other parts of the program—a key software engineering principle.
- **How They Work:**
 - Activated by a **function call**, which specifies the function’s **name** and provides **arguments** (data).
- **The Boss/Worker Analogy:**
 - A **Caller** (“Boss”) requests a task from a **Called Function** (“Worker”).
 - The “Boss” does not need to know *how* the “Worker” performs the task.
- **Function Hierarchy:**
 - Functions can call other functions, creating a chain of command where a “Worker” can also be a “Boss” to others.

Math Library Functions

- **Purpose:** Provides a collection of functions for common mathematical calculations.
 - To use them, you must include the header: **#include <math.h>**
- **Key Characteristics:**
 - Most functions take arguments of type double and return a result of type double.
 - Results can be printed directly or stored in a variable for later use.
 - Function arguments can be constants, variables, or expressions.
- **Usage Examples:**
 - Simple Call: **printf("%.2f", sqrt(900.0)); // Prints 30.00**
 - Storing the Result: **double result = sqrt(900.0);**
 - Using an Expression as an Argument: **double value = sqrt(13.0 + 3.0 * 4.0); //**
Calculates sqrt of 25.0

Function	Description	Example
<code>sqrt(x)</code>	square root of x	<code>sqrt(900.0)</code> is 30.0 <code>sqrt(9.0)</code> is 3.0
<code>cbrt(x)</code>	cube root of x (C99 and C11 only)	<code>cbrt(27.0)</code> is 3.0 <code>cbrt(-8.0)</code> is -2.0
<code>exp(x)</code>	exponential function e^x	<code>exp(1.0)</code> is 2.718282 <code>exp(2.0)</code> is 7.389056
<code>log(x)</code>	natural logarithm of x (base e)	<code>log(2.718282)</code> is 1.0 <code>log(7.389056)</code> is 2.0
<code>log10(x)</code>	logarithm of x (base 10)	<code>log10(1.0)</code> is 0.0 <code>log10(10.0)</code> is 1.0 <code>log10(100.0)</code> is 2.0
<code>fabs(x)</code>	absolute value of x as a floating-point number	<code>fabs(13.5)</code> is 13.5 <code>fabs(0.0)</code> is 0.0 <code>fabs(-13.5)</code> is 13.5
<code>ceil(x)</code>	rounds x to the smallest integer not less than x	<code>ceil(9.2)</code> is 10.0 <code>ceil(-9.8)</code> is -9.0
<code>floor(x)</code>	rounds x to the largest integer not greater than x	<code>floor(9.2)</code> is 9.0 <code>floor(-9.8)</code> is -10.0
<code>pow(x, y)</code>	x raised to power y (x^y)	<code>pow(2, 7)</code> is 128.0 <code>pow(9, .5)</code> is 3.0

Function	Description	Example
<code>fmod(x, y)</code>	remainder of x/y as a floating-point number	<code>fmod(13.657, 2.333)</code> is 1.992
<code>sin(x)</code>	trigonometric sine of x (x in radians)	<code>sin(0.0)</code> is 0.0
<code>cos(x)</code>	trigonometric cosine of x (x in radians)	<code>cos(0.0)</code> is 1.0
<code>tan(x)</code>	trigonometric tangent of x (x in radians)	<code>tan(0.0)</code> is 0.0

Principles of Good Function Design

- **Single Responsibility:** Each function should perform one single, well-defined task.
- **Descriptive Name:** The name should clearly express what the function does.
- **Decomposition:** If a function becomes too complex, break it down into smaller, simpler functions.

Square Function

```
1 // fig05_01.c
2 // Creating and using a function.
3 #include <stdio.h>
4
5 int square(int number); // function prototype
6
7 int main(void) {
8     // loop 10 times and calculate and output square of x each time
9     for (int x = 1; x <= 10; ++x) {
10         printf("%d  ", square(x)); // function call
11     }
12
13     puts("");
14 }
15
16 // square function definition returns the square of its parameter
17 int square(int number) { // number is a copy of the function's argument
18     return number * number; // returns square of number as an int
19 }
```

1	4	9	16	25	36	49	64	81	100
---	---	---	----	----	----	----	----	----	-----

Fig. 5.1 | Creating and using a function. (Part 1 of 2.)

Function Call and Definition

- **Function Call (Invocation):**

- A function is called by its name, providing arguments in parentheses.
- Example: `printf("%d ", square(x));`

- **Argument Passing:**

- A *copy* of the argument's value (x) is passed to the function's parameter (number).

- **Function Definition:**

- Defines what the function does.
- `int square(int number)` indicates it returns an int and accepts an int parameter.
- The return statement sends a value back to the caller.
- **Local Variables:** Variables defined inside a function (including parameters) are only accessible within that function.

The Function Prototype

- **What is it?** A declaration of a function that appears before it's called.
 - Example: `int square(int number);` (Note the semicolon at the end).
- **Why is it crucial?**
 - It allows the compiler to perform type checking.
 - The compiler verifies that function calls have the correct:
 - Number of arguments.
 - Type of arguments.
 - Order of arguments.
 - Usage of the return value.
- **Consistency is Key:** The prototype, function header, and function calls must all agree.

Function Structure

```
return-value-type function-name(parameter-list) {  
    // statements (the function body)  
}
```

- void is used as a return type or in the parameter list if the function doesn't return a value or take arguments.
- **Returning Control:**
 - return expression; // Returns a value.
 - return; // Returns from a void function.
 - Reaching the closing brace }.

```

5 int maximum(int x, int y, int z); // function prototype
6
7 int main(void) {
8     int number1 = 0; // first integer entered by the user
9     int number2 = 0; // second integer entered by the user
10    int number3 = 0; // third integer entered by the user
11
12    printf("%s", "Enter three integers: ");
13    scanf("%d%d%d", &number1, &number2, &number3);
14
15    // number1, number2 and number3 are arguments
16    // to the maximum function call
17    printf("Maximum is: %d\n", maximum(number1, number2, number3));
18 }
19
20 // Function maximum definition
21 int maximum(int x, int y, int z) {
22     int max = x; // assume x is largest
23
24     if (y > max) { // if y is larger than max,
25         max = y; // assign y to max
26     }
27
28     if (z > max) { // if z is larger than max,
29         max = z; // assign z to max
30     }
31
32     return max; // max is largest value
33 }

```

```
int maximum(int, int, int);
```

```
Enter three integers: 22 85 17
Maximum is: 85
```

```
Enter three integers: 47 32 14
Maximum is: 47
```

```
Enter three integers: 35 8 79
Maximum is: 79
```

Fig. 5.2 | Finding the maximum of three integers. (Part I of 2.)

Data type	printf conversion specification	scanf conversion specification
<i>Floating-point types</i>		
long double	%Lf	%Lf
double	%f	%lf
float	%f	%f
<i>Integer types</i>		
unsigned long long int	%llu	%llu
long long int	%lld	%lld
unsigned long int	%lu	%lu
long int	%ld	%ld
unsigned int	%u	%u
int	%d	%d
unsigned short	%hu	%hu
short	%hd	%hd
char	%c	%c

Arithmetic Type Conversions in C

- **Core Concept: Promotion**

- In expressions with mixed data types (e.g., `int + double`), the compiler automatically converts values to the “highest” type present.
- This is done by creating temporary copies of the lower-type values

- **Conversion to Lower Types**

- This happens only explicitly (via assignment or casting) or when passing arguments to functions.
 - **Warning:** This can cause data loss and lead to incorrect results.
 - **Example:** Passing a double 4.5 to a function expecting an int parameter results in the value 4, not 4.5. `square(4.5)` would incorrectly return 16 instead of 20.25.

Introduction to the Function-Call Stack

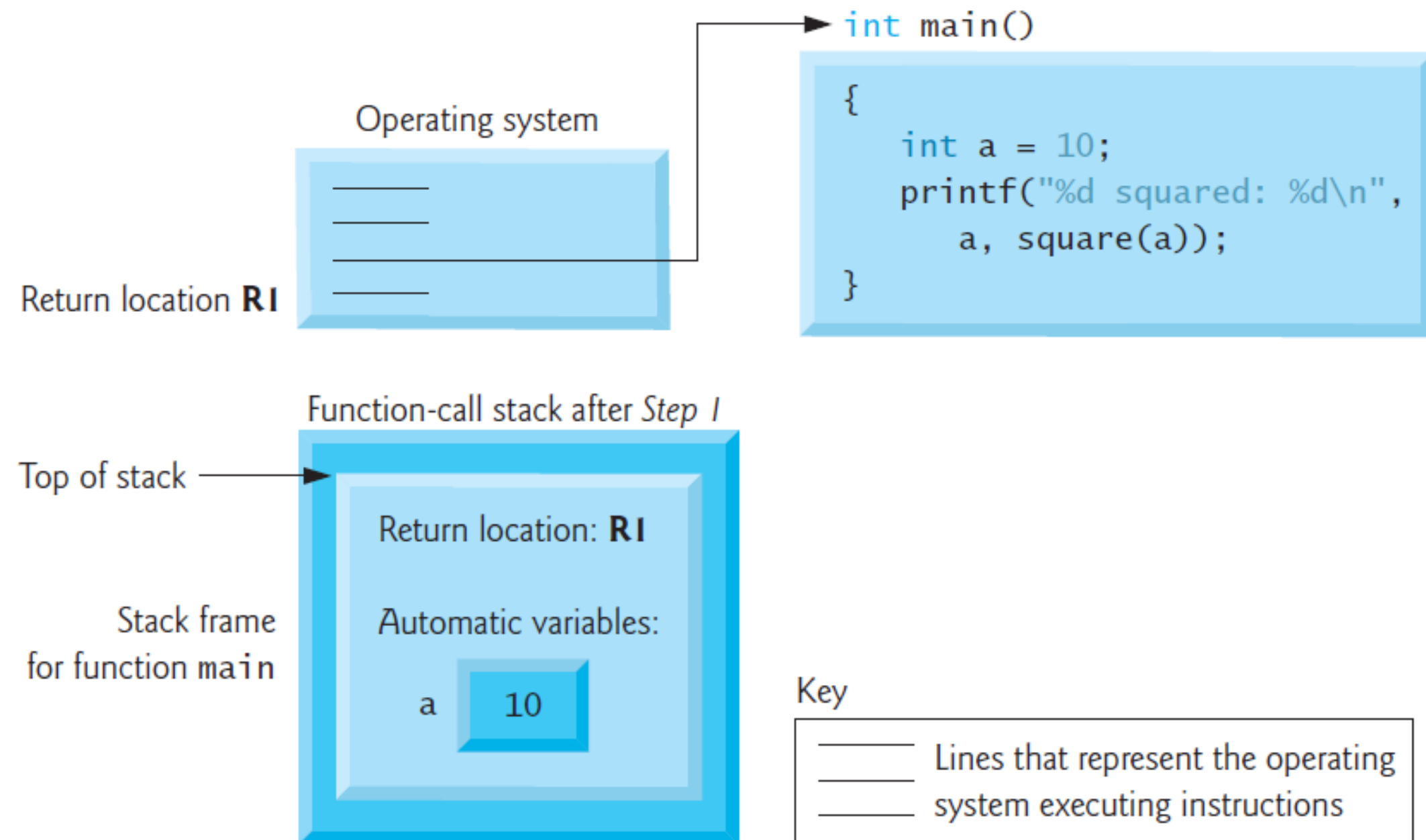
- **Core Idea:** C uses a **stack** data structure to manage function calls.
- **The Stack Analogy (A Pile of Dishes):**
 - **Push:** Placing a new item on top of the stack.
 - **Pop:** Removing the top item from the stack.
 - This is a **LIFO (Last-In, First-Out)** structure: The last function called is the first one to finish and be removed.
- **The Stack Frame:**
 - When a function is called, a **stack frame** is **pushed** onto the stack.
 - Each stack frame contains crucial information:
 - **Return Address:** Where to go back to in the calling function.
 - **Local Variables:** Memory is reserved for all local variables of the called function.
- **Stack Overflow:** A fatal error that occurs if there are too many nested function calls, causing the stack to run out of memory.

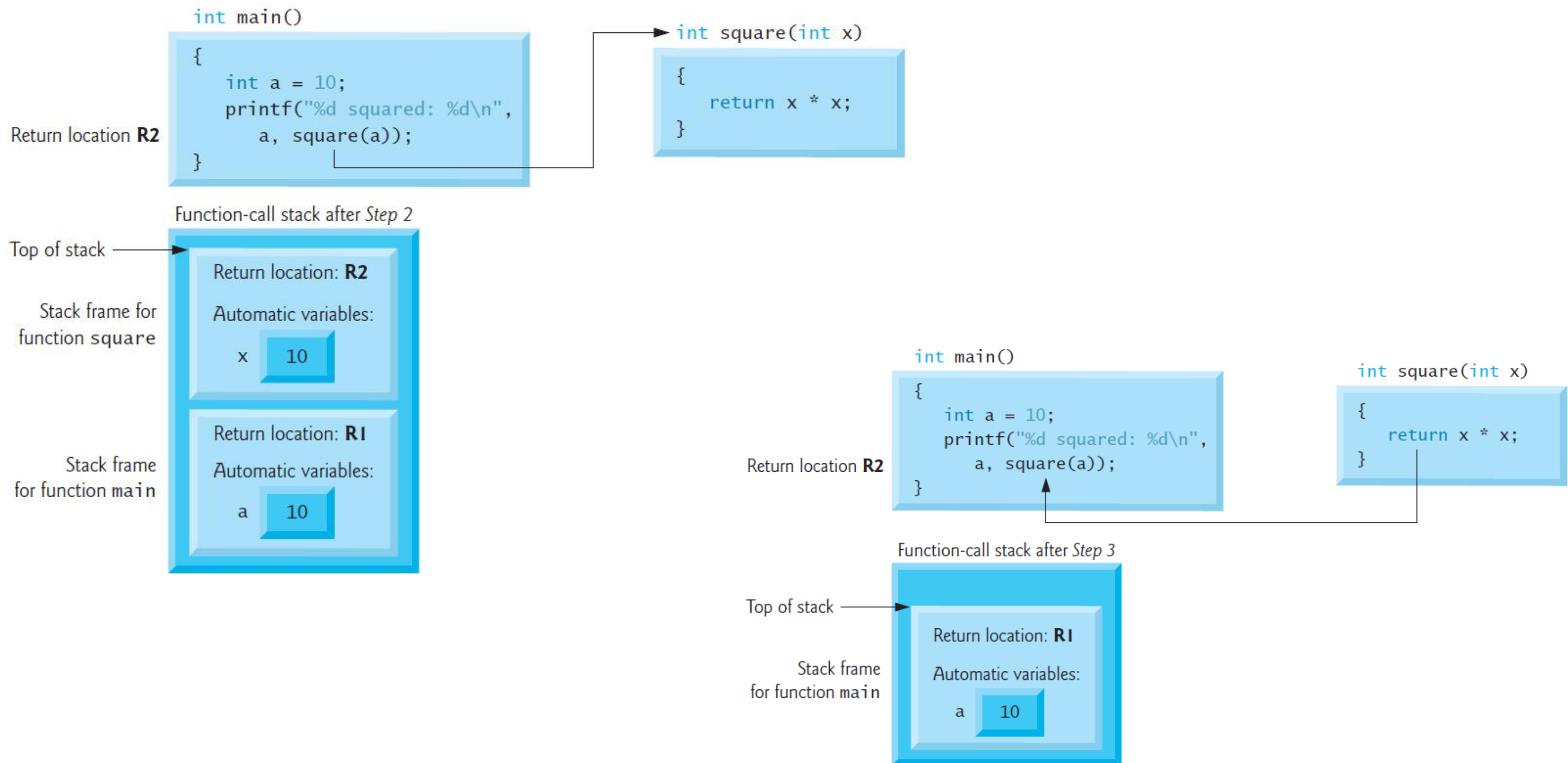
```

1 // fig05_03.c
2 // Demonstrating the function-call stack
3 // and stack frames using a function square.
4 #include <stdio.h>
5
6 int square(int x); // prototype for function
7
8 int main() {
9     int a = 10; // value to square (local vari
10
11     printf("%d squared: %d\n", a, square(a));
12 }
13
14 // returns the square of an integer
15 int square(int x) { // x is a local variable
16     return x * x; // calculate square and retu
17 }

```

10 squared: 100





The Call Stack in Action (A Simple Example)

- This slide illustrates the process of main calling a function named square.
- **OS Calls main**
 - main's stack frame is **pushed** onto the stack.
 - It contains the return address to the Operating System and space for main's variables.
- **main Calls square**
 - square's stack frame is **pushed** on top of main's.
 - It contains the return address to main and space for square's variables.
 - The program is now executing inside square.
- **square Returns**
 - square's stack frame is **popped** from the stack.
 - Its local variables are destroyed.
 - Control returns to the address in main that was stored in the frame.
- **main Returns**
 - main's stack frame is **popped**.
 - Control returns to the Operating System.

Random-Number Generation

- **Core Function:** `rand()`
 - Found in the `<stdlib.h>` header.
 - Generates a pseudorandom integer between 0 and `RAND_MAX` (a constant that is at least 32,767).
- **The Problem:** The raw output of `rand()` is often not in the range you need (e.g., for a six-sided die, you need 1-6).
- **The Solution: Scaling and Shifting**
 - **Scaling:** Use the modulo operator (%) to limit the upper bound.
 - `rand() % 6` produces a random integer from 0 to 5.
 - **Shifting:** Use addition (+) to change the starting point of the range.
 - `1 + rand() % 6` shifts the range to produce a random integer from 1 to 6.

```
1 // fig05_04.c
2 // Shifted, scaled random integers produced by 1 + rand() % 6.
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(void) {
7
8     for (int i = 1; i <= 10; ++i) {
9         printf("%d  ", 1 + (rand() % 6)); // display random die value
10    }
11
12    puts("");
13 }
```

6 6 5 5 6 5 1 1 5 3

```

14 // loop 60000000 times and summarize results
15 for (int roll = 1; roll <= 60000000; ++roll) {
16     int face = 1 + rand() % 6; // random number from 1 to 6
17
18     // determine face value and increment appropriate counter
19     switch (face) {
20         case 1: // rolled 1
21             ++frequency1;
22             break;
23         case 2: // rolled 2
24             ++frequency2;
25             break;
26         case 3: // rolled 3
27             ++frequency3;
28             break;
29         case 4: // rolled 4
30             ++frequency4;
31             break;
32         case 5: // rolled 5
33             ++frequency5;
34             break;
35         case 6: // rolled 6
36             ++frequency6;
37             break; // optional
38     }

```

Face	Frequency
1	9999294
2	10002929
3	9995360
4	10000409
5	10005206
6	9996802

Fig. 5.5 | Rolling a six-sided die 60,000,000 times. (Part 2 of 2.)

The Problem of Repeatability and Seeding

- **The Issue:** `rand()` is **pseudorandom**. By default, it will produce the *exact same sequence* of “random” numbers every time you run the program.
 - This is intentional and useful for debugging purposes.
- **The Solution: Seeding with `srand()`**
 - To get a different sequence on each run, you must “seed” the random number generator.
 - The `srand()` function (from `<stdlib.h>`) is used for this. It takes an integer “seed” to start the sequence.
 - Different seeds produce different sequences.

```

8  int seed = 0; // number used to seed the random-number generator
9  scanf("%d", &seed);
10
11  srand(seed); // seed the random-number generator
12
13  for (int i = 1; i <= 10; ++i) {
14      printf("%d  ", 1 + (rand() % 6)); // display random die value
15  }
16
17  puts("");

```

```

Enter seed: 67
6  1  4  6  2  1  6  1  6  4

```

```

Enter seed: 867
2  4  6  1  6  1  1  3  6  2

```

```

Enter seed: 67
6  1  4  6  2  1  6  1  6  4

```

Fig. 5.6 | Randomizing the die-rolling program. (Part 2 of 2.)

Automating the Seed:

- To avoid manually entering a new seed every time, use the system clock.
- The standard practice is to use this line once at the start of your program:

```
srand(time(NULL)); // Requires the <time.h> header
```

- time(NULL) returns the current time in seconds, providing a unique seed for each execution.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {

    srand(time(NULL));
    for (int j=1; j<=100;j++)
    {

    }
    for (int i = 1; i <= 10; ++i) {
        printf("%d  ", 1 + (rand() % 6)); // display random die value
    }

    puts("");
}

```

```
6  6  5  4  3  6  2  2  5  2
```

```
Process returned 0 (0x0)    execution time : 0.020 s
```

```
Press any key to continue.
```

```
|
```

The Generalized Formula for Random Numbers

```
int n = a + rand() % b;
```

- n: The final random number.
- a: The **shifting value**. This is the **first number** in your desired range.
- b: The **scaling factor**. This is the **width** of your range (i.e., how many different numbers you want to generate).

The Game of Craps: Rules Summary

Objective: The player rolls two six-sided dice, and the outcome is determined by the sum of the roll.

- **Phase 1: The First Throw (The “Come-Out Roll”)**
- **INSTANT WIN :** If the sum is **7** or **11**.
- **INSTANT LOSE :** If the sum is **2, 3, or 12**.
- **ESTABLISH A “POINT”**
 - If the sum is **4, 5, 6, 8, 9, or 10**.
 - This number becomes your **“Point”**, and the game continues to Phase 2.
- **Phase 2: Making the Point** (*This phase only happens if a “Point” was established in Phase 1*)
- **WIN Condition:**
 - You must roll your **“Point”** number again.
- **LOSE Condition:**
 - You lose if you roll a **7** *before* making your Point.

```

7  enum Status {CONTINUE, WON, LOST}; // constants represent game status
8
9  int rollDice(void); // rollDice function prototype
10
11 int main(void) {
12     srand(time(NULL)); // randomize based on current time
13
14     int myPoint = 0; // player must make this point to win
15     enum Status gameStatus = CONTINUE; // may be CONTINUE, WON, or LOST
16     int sum = rollDice(); // first roll of the dice
17
18     // determine game status based on sum of dice
19     switch(sum) {
20         // win on first roll
21         case 7: // 7 is a winner
22             case 11: // 11 is a winner
23                 gameStatus = WON;
24                 break;
25         // lose on first roll
26         case 2: // 2 is a loser
27         case 3: // 3 is a loser
28         case 12: // 12 is a loser
29             gameStatus = LOST;
30             break;
31         // remember point
32         default:
33             gameStatus = CONTINUE; // player should keep rolling
34             myPoint = sum; // remember the point
35             printf("Point is %d\n", myPoint);
36             break; // optional
37     }

```

```

38
39     // while game not complete
40     while (CONTINUE == gameStatus) { // player should keep rolling
41         sum = rollDice(); // roll dice again
42
43         // determine game status
44         if (sum == myPoint) { // win by making point
45             gameStatus = WON;
46         }
47         else if (7 == sum) { // lose by rolling 7
48             gameStatus = LOST;
49         }
50     }
51

```

```

52 // display won or lost message
53 if (WON == gameStatus) { // did player win?
54     puts("Player wins");
55 }
56 else { // player lost
57     puts("Player loses");
58 }
59 }
60
61 // roll dice, calculate sum and display results
62 int rollDice(void) {
63     int die1 = 1 + (rand() % 6); // pick random die1 value
64     int die2 = 1 + (rand() % 6); // pick random die2 value
65
66     // display results of this roll
67     printf("Player rolled %d + %d = %d\n", die1, die2, die1 + die2);
68     return die1 + die2; // return sum of dice
69 }

```

Player wins on the first roll:

Player rolled 5 + 6 = 11
Player wins

Player wins on a subsequent roll:

Player rolled 4 + 1 = 5
Point is 5
Player rolled 6 + 2 = 8
Player rolled 2 + 1 = 3
Player rolled 3 + 2 = 5
Player wins

Player loses on the first roll:

Player rolled 1 + 1 = 2
Player loses

Player loses on a subsequent roll:

Player rolled 6 + 4 = 10
Point is 10
Player rolled 3 + 4 = 7
Player loses

Enumeration

- **What It Is:** A custom type that replaces numbers with descriptive names.
- **Key Benefit:** Improves code readability.
 - `gameStatus = WON;` is much clearer than `gameStatus = 1;`.
- **How It Works:** The compiler assigns integers automatically, starting from 0
- **Convention:** Use all **UPPERCASE** letters for enum constants to indicate they are constants, not variables.

```
// Defines a new type named 'enum Status'  
enum Status { CONTINUE, // Value is 0  
              WON,      // Value is 1  
              LOST };   // Value is 2
```

```
// Declaration  
enum Status gameStatus;  
  
// Usage  
gameStatus = CONTINUE; // Start the game  
// ... later in the game  
gameStatus = WON;      // If the player wins
```

Storage Classes

- **Identifiers** are names for variables and functions.
- Beyond a **name** and **type**, every identifier in C has other key attributes:
 - **Storage Duration:** How long it exists in memory.
 - **Scope:** Where it can be accessed in the code.
- These attributes are controlled by **Storage-Class Specifiers**:
 - auto
 - static

Automatic Storage Duration (The Default)

- **Keyword:** `auto` (but is rarely typed, as it's the default for local variables).
- **Lifetime: Temporary.**
 - Variables are created when program control enters their block (e.g., a function).
 - They are **destroyed** when the block is exited.
- **Behavior:** The variable **loses its value** between function calls.
- **Who it applies to:** Local variables declared inside a function or its parameter list.
- **Benefit:** This is the most common behavior and it conserves memory.

```
void myFunction() {  
    int x = 10; // This is an 'auto' variable.  
               // It is created and set to 10 each time the function is called.  
} // 'x' is destroyed here.
```

Static Storage Duration

- **Keywords:** static
- **Lifetime: Permanent.**
 - Variables exist from the moment the program begins execution until it terminates.
 - Storage is allocated and initialized **only once**.
- **Two Main Types:**
 - **Global Variables** (declared outside any function).
 - **Static Local Variables** (declared inside a function with the static keyword).

Static Storage: Global Variables

- **How to create:** Declare a variable *outside* of any function.
- **Scope:** They can be referenced by any function that follows their declaration in the file.
- **Behavior:** They retain their value throughout the entire program execution.
-  **CAUTION: AVOID GLOBAL VARIABLES**
 - **Problem:** Any function can modify a global variable. This can lead to unexpected and hard-to-find bugs called “unintended side effects.”
 - **Best Practice:** Variables should be local to the functions that use them.

```
int globalCounter = 0; // A global variable. Avoid this pattern.
```

```
void functionA() {  
    globalCounter++;  
}
```

```
void functionB() {  
    globalCounter = 100; // Unintended side effect?  
}
```

Static Storage: Static Local Variables

- **How to create:** Use the static keyword for a variable declared *inside* a function.
- **The Key Feature:** It **retains its value** between function calls.
- **Initialization:** It is initialized **only once** when the program starts. (If not specified, numeric types default to 0).
- **Scope:** It is still a **local** variable; it can only be accessed within the function where it is defined.

```
void counterFunction() {  
    // This initialization happens only ONCE  
    static int callCount = 0;  
  
    ● callCount++;  
    printf("This function has been called %d time(s).\n", callCount);  
}  
  
// Calling counterFunction() multiple times will print:  
// "This function has been called 1 time(s)."  
// "This function has been called 2 time(s)."  
// "This function has been called 3 time(s)."
```

```

3 #include <stdio.h>
4
5 void useLocal(void); // function prototype
6 void useStaticLocal(void); // function prototype
7 void useGlobal(void); // function prototype
8
9 int x = 1; // global variable
10
11 int main(void) {
12     int x = 5; // local variable to main
13
14     printf("local x in outer scope of main is %d\n", x);
15
16     { // start new scope
17         int x = 7; // local variable to new scope
18
19         printf("local x in inner scope of main is %d\n", x);
20     } // end new scope
21
22     printf("local x in outer scope of main is %d\n", x);
23
24     useLocal(); // useLocal has automatic local x
25     useStaticLocal(); // useStaticLocal has static local x
26     useGlobal(); // useGlobal uses global x
27     useLocal(); // useLocal reinitializes automatic local x
28     useStaticLocal(); // static local x retains its prior value
29     useGlobal(); // global x also retains its value
30
31     printf("\nlocal x in main is %d\n", x);
32 }

```

```

local x in outer scope of main is 5
local x in inner scope of main is 7
local x in outer scope of main is 5

local x in useLocal is 25 after entering useLocal
local x in useLocal is 26 before exiting useLocal

local static x is 50 on entering useStaticLocal
local static x is 51 on exiting useStaticLocal

global x is 1 on entering useGlobal
global x is 10 on exiting useGlobal

local x in useLocal is 25 after entering useLocal
local x in useLocal is 26 before exiting useLocal

local static x is 51 on entering useStaticLocal
local static x is 52 on exiting useStaticLocal

global x is 10 on entering useGlobal
global x is 100 on exiting useGlobal

local x in main is 5

```

```

34 // useLocal reinitializes local variable x during each call
35 void useLocal(void) {
36     int x = 25; // initialized each time useLocal is called
37
38     printf("\nlocal x in useLocal is %d after entering useLocal\n", x);
39     ++x;
40     printf("local x in useLocal is %d before exiting useLocal\n", x);
41 }
42
43 // useStaticLocal initializes static local variable x only the first time
44 // the function is called; value of x is saved between calls to this
45 // function
46 void useStaticLocal(void) {
47     static int x = 50; // initialized once
48
49     printf("\nlocal static x is %d on entering useStaticLocal\n", x);
50     ++x;
51     printf("local static x is %d on exiting useStaticLocal\n", x);
52 }
53
54 // function useGlobal modifies global variable x during each call
55 void useGlobal(void) {
56     printf("\nglobal x is %d on entering useGlobal\n", x);
57     x *= 10;
58     printf("global x is %d on exiting useGlobal\n", x);
59 }

```

Recursion

- **What is a Recursive Function?**

- A function that calls itself, either directly or indirectly.

- **The Core Idea:**

- Solve a large problem by breaking it down into smaller, simpler versions of the *exact same problem*.
- This process repeats until the problem becomes so simple it can be solved directly.

The Two Essential Components of Recursion

- Every recursive function is built from two critical parts:
- **1. The Base Case (The Stopping Condition)**
 - The simplest possible version of the problem that the function can solve directly, without calling itself again.
 - **This is crucial!** Without a base case, a recursive function would call itself forever, leading to a program crash (a “stack overflow”).
- **2. The Recursive Step (The Simplification)**
 - The function is called with a more complex problem. It does two things:
 - Performs a small piece of work.
 - Calls itself with a *slightly simpler or smaller* version of the problem.
 - Each recursive call must bring the problem closer to the base case.

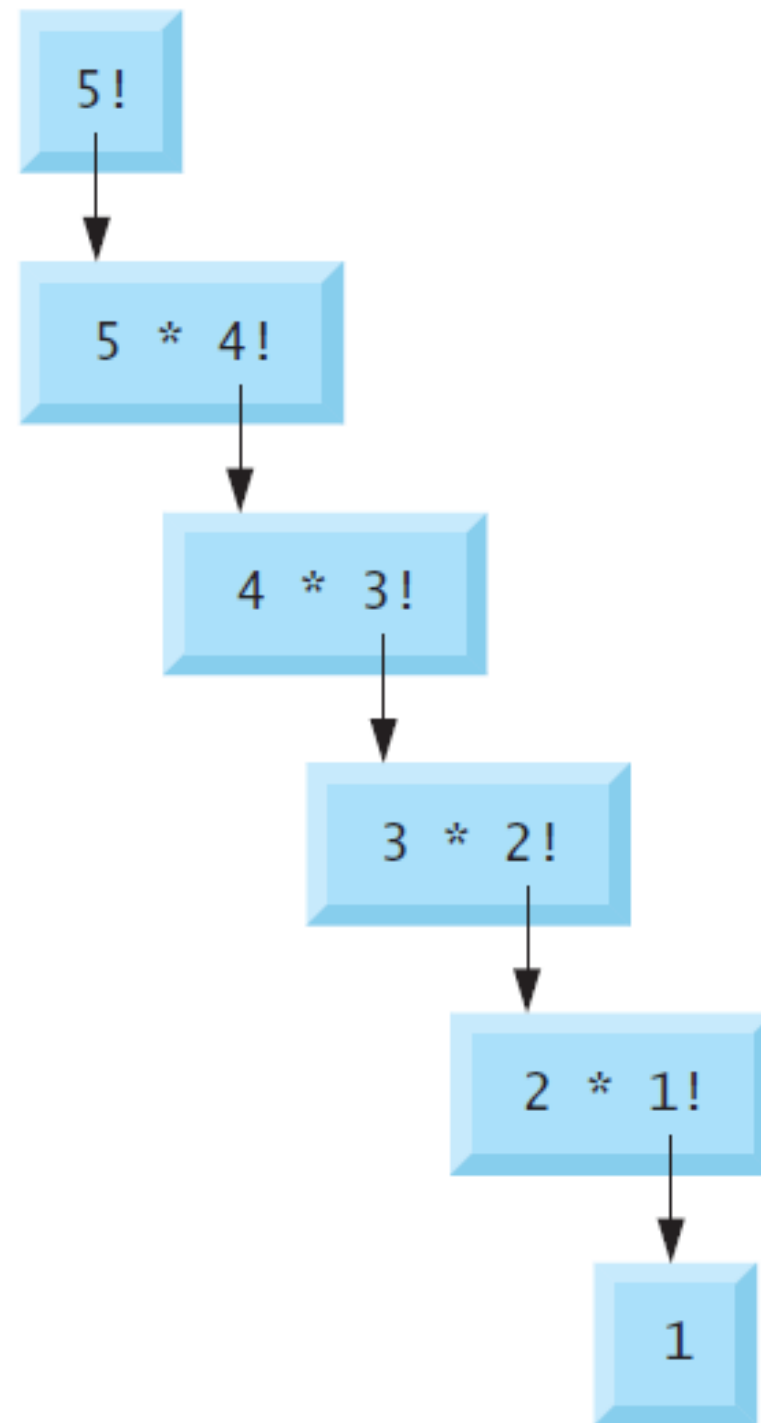
Case Study - Calculating a Factorial (n!)

- **Mathematical Definition:** The factorial of a non-negative integer n is the product of all positive integers less than or equal to n .
 - $n! = n * (n - 1) * (n - 2) * ... * 1$
- **Example:**
 - $5! = 5 * 4 * 3 * 2 * 1 = 120$
- **Special Cases (Our future Base Cases):**
 - $1! = 1$
 - $0! = 1$ (by definition)

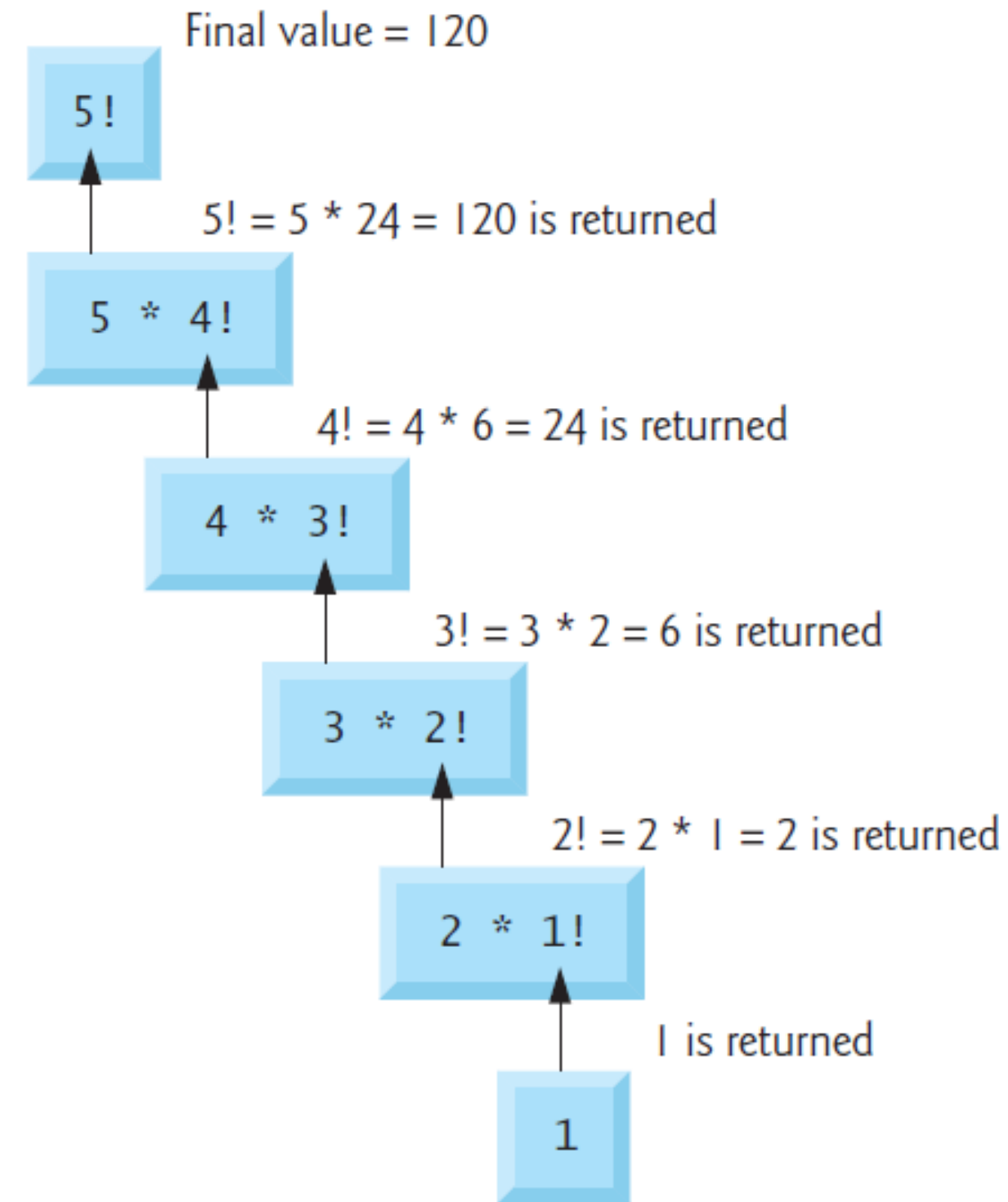
The Recursive Insight for Factorial

- The key is to spot the recursive relationship:
 - $5! = 5 * (4 * 3 * 2 * 1)$ which is the same as $5! = 5 * 4!$
 - Generalizing this gives us the formula: $n! = n * (n - 1)!$
- **This formula defines our two recursive components:**
 - **Recursive Step:** The factorial of n is n times the factorial($n - 1$).
 - **Base Case:** We know the factorial of 1 is 1. This is our stopping point.

a) Sequence of recursive calls



b) Values returned from each recursive call



```

1 // fig05_09.c
2 // Recursive factorial function.
3 #include <stdio.h>
4
5 unsigned long long int factorial(int number);
6
7 int main(void) {
8     // calculate factorial(i) and display result
9     for (int i = 0; i <= 21; ++i) {
10         printf("%d! = %llu\n", i, factorial(i));
11     }
12 }
13
14 // recursive definition of function factorial
15 unsigned long long int factorial(int number) {
16     if (number <= 1) { // base case
17         return 1;
18     }
19     else { // recursive step
20         return (number * factorial(number - 1));
21     }
22 }

```

```

0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
11! = 39916800
12! = 479001600
13! = 6227020800
14! = 87178291200
15! = 1307674368000
16! = 20922789888000
17! = 355687428096000
18! = 6402373705728000
19! = 121645100408832000
20! = 2432902008176640000
21! = 14197454024290336768

```

Fig. 5.9 | Recursive factorial function. (Part 2 of 2.)

Fig. 5.9 | Recursive factorial function. (Part 1 of 2.)

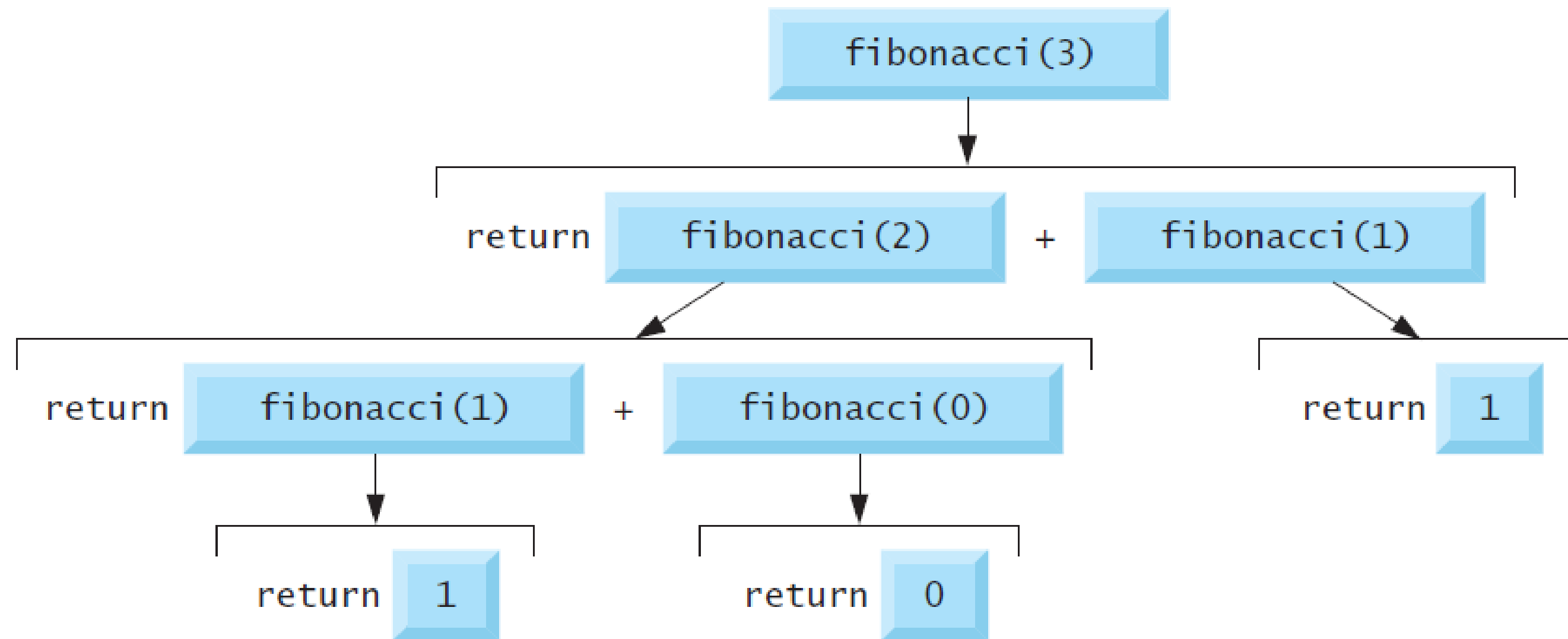
Example Using Recursion: Fibonacci Series

- **What is it?** A series of numbers with a simple, elegant property.
 - It starts with 0 and 1.
 - Each subsequent number is the **sum of the two previous numbers**.
- **The Sequence:**
- 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

The Recursive Definition

- The Fibonacci series can be defined perfectly using recursion.
- **Base Cases (The Stopping Points):**
 - $\text{fibonacci}(0) = 0$
 - $\text{fibonacci}(1) = 1$
- **Recursive Step (The General Rule):**
 - For any number n greater than 1:
 - $\text{fibonacci}(n) = \text{fibonacci}(n - 1) + \text{fibonacci}(n - 2)$
- This definition translates directly and elegantly into a recursive function.

Visualizing the Recursive Calls for fibonacci(3)



```

1 // fig05_10.c
2 // Recursive fibonacci function.
3 #include <stdio.h>
4
5 unsigned long long int fibonacci(int n); // function prototype
6
7 int main(void) {
8     // calculate and display fibonacci(number) for 0-10
9     for (int number = 0; number <= 10; number++) {
10         printf("Fibonacci(%d) = %llu\n", number, fibonacci(number));
11     }
12
13     printf("Fibonacci(20) = %llu\n", fibonacci(20));
14     printf("Fibonacci(30) = %llu\n", fibonacci(30));
15     printf("Fibonacci(40) = %llu\n", fibonacci(40));
16 }
17
18 // Recursive definition of function fibonacci
19 unsigned long long int fibonacci(int n) {
20     if (0 == n || 1 == n) { // base case
21         return n;
22     }
23     else { // recursive step
24         return fibonacci(n - 1) + fibonacci(n - 2);
25     }
26 }

```

```

Fibonacci(0) = 0
Fibonacci(1) = 1
Fibonacci(2) = 1
Fibonacci(3) = 2
Fibonacci(4) = 3
Fibonacci(5) = 5
Fibonacci(6) = 8
Fibonacci(7) = 13
Fibonacci(8) = 21
Fibonacci(9) = 34
Fibonacci(10) = 55
Fibonacci(20) = 6765
Fibonacci(30) = 832040
Fibonacci(40) = 102334155

```

Fig. 5.10 | Recursive fibonacci function.

Recursion vs. Iteration: The Two Faces of Repetition

- **The Goal:** Both recursion and iteration are methods for performing repetitive tasks in a program.
- **The Methods:**
 - **Iteration:** Uses a loop (like for or while) with a counter or condition to repeat a block of code.
 - **Recursion:** A function repeatedly calls itself, solving a smaller version of the problem each time until it reaches a final “base case”.
- **The Key Question:** Both can solve the same problems, so when should you choose one over the other?

The Case AGAINST Recursion: The Performance Cost

- While powerful, recursion has significant downsides.
- **1. Function Call Overhead:**
 - Every function call consumes processor time (setting up, tearing down). Recursion involves many function calls, which can be slower than a simple loop.
- **2. Memory Consumption:**
 - Each recursive call creates a new copy of the function's local variables in memory (on the "call stack").
 - For a deep recursion (many calls), this can consume a large amount of memory. An infinite recursion will lead to a **stack overflow** crash.
- **The Bottom Line:** For raw performance and memory efficiency, iteration is almost always superior.

The Case FOR Recursion: When to Choose It

- If iteration is faster, why ever use recursion?
- **1. Natural Problem Mapping:**
 - For some problems, the recursive approach perfectly mirrors the real-world or mathematical definition. The code becomes incredibly intuitive and elegant.
 - *Examples: Navigating a file system, traversing tree-like data structures, many mathematical formulas (Factorial, Fibonacci).*
- **2. Clarity and Simplicity:**
 - A recursive solution can be far easier to write, understand, and debug than a complex, non-obvious iterative solution.
- **Key Principle:** Choose recursion when the clarity and simplicity of the solution outweigh the potential performance cost.

The Bigger Picture: A Software Engineering Trade-Off

- The choice between recursion and iteration is part of a larger discussion in programming.
- **Good Software Engineering:**
 - Favors breaking large problems into smaller, manageable functions. This makes code easier to test, debug, and maintain.
 - This approach aligns with recursion's philosophy of breaking down problems.
- **High Performance:**
 - Focuses on raw speed and efficiency. This often means avoiding the overhead of function calls.
 - This approach aligns with iteration's efficiency.
- **Modern Conclusion:** For the vast majority of applications, **good software engineering is more important.** Hardware is incredibly fast. Choose the solution (recursive or iterative) that makes your code the most understandable and correct. Only optimize for performance when it is absolutely necessary.